Exercise 7 - 8./9.12.2025

Learning Goals

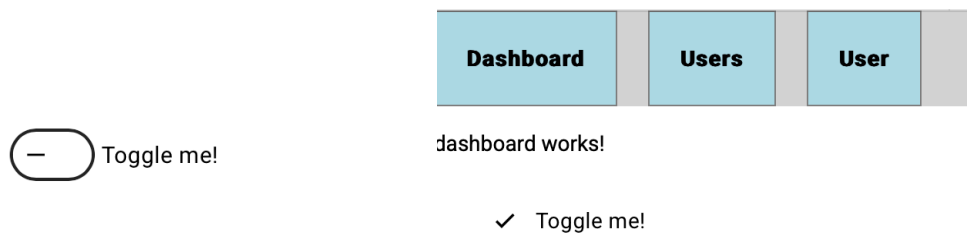
Today we want to get familiar using **external UI libraries** like **Angular Material** and selecting our own theme for customization.
Depending on the framework there might exist lots of component libraries (like vuetify for Vue or PrimeNG for Angular). They might be more or less invasive (means removing or replacing them is different in regards of efforts). Usually the biggest challenge is installing and configuring the library and to understand how components are used / what wrapping code is needed. We will practice this today.

Practical Exercises

1) **Install Angular Material** in your app. Ensure if you have Angular v20, you install material v20 => ng add @angular/material@20

If you use Angular v18 => ng add @angular/material@18
Pick any starter theme you like.
Which files get modified? Is there an Angular package being installed? Does this give a hint why we need matching versions of Material and Angular?
Look at the changes caused by the schematics scripts that ran on command line command execution.
Follow the docu how to use the **<mat-slide-toggle>** on any screen and use it in the dashboard template.
Ensure you look at the docu for the correct version (top right):
https://material.angular.io/guide/getting-started

Hint: you will have to stop and restart the app, hot reload is not be able to add new imported modules. Result might look like this:



2) Customize the colours to match the HS Fulda theming (e.g. using the green = #66cc33 as primary colour). Create a theme and select the HS green as primary color: ***ng generate @angular/material:m3-theme***

A SCSS file should have been created. Remove the theme that got installed in 1) and replace it with the new one.

Docu: https://material.angular.io/guide/theming#defining-a-theme

Hint: the docu is not easy to understand, the created scss files contains palettes. These palettes can be used to define a theme (e.g. in styles.scss):

```
@use 'theme-colors' as custom-theme;

html {
$custom-theme: mat.define-theme((
      color: (
        primary: custom-theme.$primary-palette,
        tertiary: custom-theme.$tertiary-palette,
      )
    ));

@include mat.all-component-themes($custom-theme);
}
```

Now the colors of your toggle should change to shades of green:





3) Download or screenshot the logo of HS Fulda and add it to the left corner of your header. Create a header if you do not have one already and use it in your app.component.html template. Hint: the logo goes into the ***public/*** folder to be served via localhost:4200/IMAGE_NAME. Scale the img to match a fixed height of your header.

It might look similar to this:



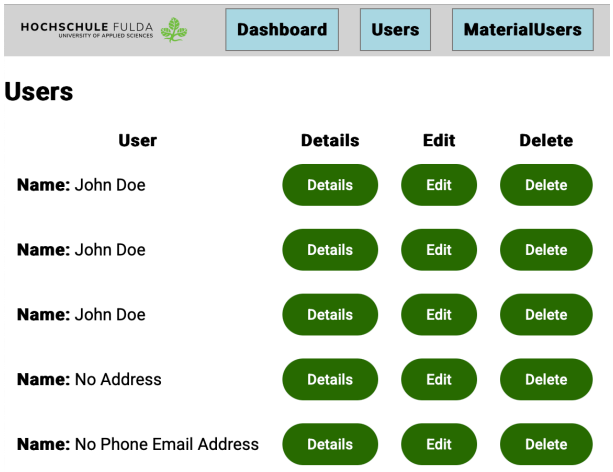
4) Create a new component using schematics named **MaterialUsersComponent**. Link that component in your header. This will be a copy of your UsersComponent with a list of users and the (link) buttons. Replace all the buttons in the copied HTML template of the new component. with Angular Material Buttons. If you have anchor links (<a ...>) you can use the ***MatAnchor*** import to make them look like a button without needing to wrap them in a <button> (<a matButton="filled" ...> or <a mat-flat-button ... >)! It might be needed to remove all/some CSS for the button or anchor element that would override the Material design theme.

Do you see an advantage of a directive compared to a component now?

See https://material.angular.io/components/button/overview

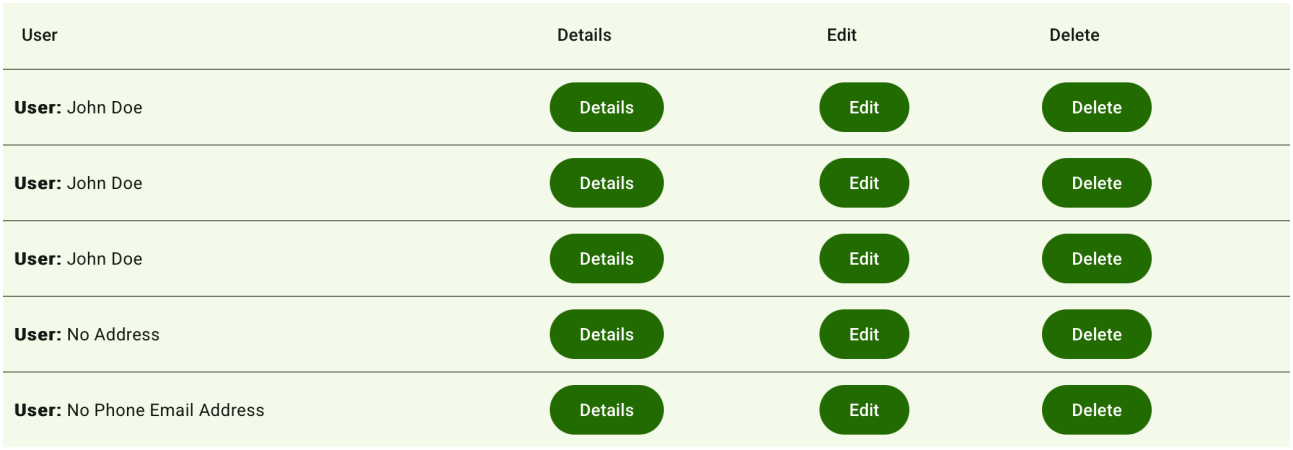
What did you need to import where to use the directive in a template?

Result might look similar to this (I removed the „User“ from the header and it is useless without the path param for the id):



5) Now replace the table in your MaterialUsersComponent with Angular Material Tables (***mat-table***) or use a table if you did not have one yet.
=> https://material.angular.io/components/table/overview
What did you need to import (there will be multiple imports)? Did sth need to be changed substantially in HTML and/or TS?

Result might look similar to this:



6) Now create a new component ***MaterialUserEditComponent*** which is copying the UserEditComponent contents. It should be linked if you click on „edit“ in the ***MaterialUsersComponent*** template. Replace all input fields in the form with Angular Material Form Fields. You will need 3 different imports from Angular Material to get this fully functional.
Combine your form-fields with the directive ***matInput*** at <input>s and replace <label> with <***mat-label***>. Wrap everything in a <***mat-form-field***>. The Angular logic using the FormsModule can stay as is for a template driven form.
Docu: https://material.angular.io/components/form-field/overview
Hint: the component is quite sensitive / strict in regards of undefined checks, you might use some @if (...) wrapping logic to ensure safety, or: pre-populate a default user with all mandatory attributes in the TS, that the template can always bind all values via ngModel.

What do you observe in terms of effort for the forms? Was it less or more complex to modify the template compared to the table? What did you need to import? Did sth need to be changed substantially in HTML or TS?

Result might look similar to this:

